Showcasing SmartRetail on GitHub — Resume & LinkedIn Guide

How to publish this project professionally so recruiters and hiring managers can find it and understand what you built.

Part 1 — Setting Up the GitHub Repository

1.1 Create a GitHub Account (if you don't have one)

Go to github.com and sign up. Use your real name — this is professional.

1.2 Create a New Repository

1. Click the + icon (top right) → **New repository**
2. Fill in:
   - **Repository name:** `smartretail-pos-pipeline`
   - **Description:** `End-to-end retail POS data pipeline: Python → Databricks (Bronze/Silver) → dbt Core (Gold) | Delta Lake | Data Quality`
   - **Visibility:** Public (so recruiters can see it)
   - **Add a README file:** check this box
3. Click **Create repository**

1.3 Connect Your Local Project to GitHub

In VS Code terminal (in your `smartretail` folder):

```powershell
git init
git remote add origin https://github.com/YOUR-USERNAME/smartretail-pos-pipeline.git
```

1.4 What to Include / Exclude

Your `.gitignore` already handles this but double-check before pushing. Never commit:

- `.env` (contains your token)
- `venv/` folder
- `data/` folder (large generated files)
- `dbt_smartretail/target/` (compiled output)

Safe to commit:

- All Python scripts in `notebooks/`
- All dbt SQL models
- All YAML config files
- `requirements.txt`
- `.env.template`

1.5 Push Your Code

```powershell
git add .
git commit -m "Initial commit: SmartRetail POS pipeline with Databricks + dbt Core"
git push -u origin main
```

Part 2 — Writing a Strong README.md

The README is the first thing recruiters see. Replace the default one with this template — fill in the sections marked with `[ ]`:

```markdown
# SmartRetail POS Intelligence Pipeline

An end-to-end data engineering pipeline that processes synthetic retail
point-of-sale data through a Bronze → Silver → Gold medallion architecture
using Databricks Free Edition and dbt Core.

## Stack
- **Python** — synthetic data generation with Faker
- **Databricks Free Edition** — Bronze ingestion and Silver cleaning (PySpark + Delt
- **dbt Core** — Gold analytical models with automated data quality tests
- **Delta Lake** — storage format throughout
- **Unity Catalog** — data governance and table management

## Architecture
```

Python (Faker) → Local Parquet → Databricks Volume
↓
Bronze Layer (raw Delta tables)
↓

Silver Layer (cleaned + validated)
↓
dbt Core → Gold Layer (6 analytical models)

```
## What It Does

Simulates a retail chain with 20 stores, 500 products, and 10,000 customers
generating ~5,000 transactions per day. Intentional data quality issues are
injected (nulls, duplicates, invalid IDs, future dates) and caught during
the Silver cleaning stage.

## Gold Models

| Model | Description |
|---|---|
| `gold_sales_daily` | Daily sales by store, product, and payment method |
| `gold_store_performance` | Store-level KPIs — revenue, transactions, basket size
|
| `gold_product_performance` | Product revenue, margin, and discount analysis |
| `gold_customer_lifetime_value` | Customer segmentation by spend and purchase
frequency |
| `gold_inventory_risk_daily` | Stockout risk classification per store/product |
| `gold_promotion_effectiveness` | Promotion revenue impact and discount cost |

## Data Quality

53 automated dbt tests across all Gold models covering:
- Not null constraints on key columns
- Uniqueness checks on primary keys
- Accepted value validation (regions, payment methods, categories)
- Numerical range checks (positive sales, non-negative stock)

## How to Run

1. Clone the repo
2. Create a virtual environment: `python -m venv venv`
3. Install dependencies: `pip install -r requirements.txt`
4. Copy `.env.template` to `.env` and fill in your Databricks credentials
5. Generate data: `python notebooks/00_generate_dirty_pos_data.py`
6. Upload to Databricks Volume and run Bronze/Silver notebooks
7. Run dbt: `cd dbt_smartretail && dbt build`

## Project Structure
```

```
smartretail/
├── notebooks/
```

```
│   ├── 00_generate_dirty_pos_data.py  # data generator
│   ├── 01_bronze_ingestion.py       # Databricks Bronze notebook
│   └── 02_silver_cleaning.py        # Databricks Silver notebook
├── dbt_smartretail/
│   ├── models/
│   │   ├── sources.yml
│   │   └── gold/                # 6 Gold models + schema tests
│   ├── dbt_project.yml
│   └── packages.yml
├── data/configs/                # pipeline configuration files
├── .env.template
└── requirements.txt
```

Copy this into your `README.md` file, save it, then commit and push:

```powershell
git add README.md
git commit -m "Add project README"
git push
```

Part 3 — Making It Look Professional on GitHub

Add Topics/Tags to Your Repository

On your GitHub repo page:

1. Click the gear icon next to **About**
2. Add topics: `data-engineering`, `dbt`, `databricks`, `pyspark`, `delta-lake`, `python`, `etl`, `medallion-architecture`
3. Click **Save changes**

These tags help your repo show up in searches.

Pin the Repository on Your Profile

1. Go to your GitHub profile page
2. Click **Customize your pins**
3. Select `smartretail-pos-pipeline`
4. Click **Save pins**

Now it's the first thing people see when they visit your profile.

Part 4 — Resume Bullet Points

Use these as a starting point and adjust to match your experience:

Data Engineer / Data Analytics role:

> Built an end-to-end retail POS data pipeline using Python, PySpark, Databricks, and dbt Core implementing a Bronze/Silver/Gold medallion architecture with Delta Lake storage and 53 automated data quality tests

> Designed and implemented 6 dbt Gold analytical models covering sales, inventory risk, customer lifetime value, and promotion effectiveness; configured dbt-databricks adapter for Databricks Unity Catalog integration

> Engineered a data quality framework using dbt schema tests to validate 7 Silver entities against null, uniqueness, referential integrity, and business rule constraints with automated rejection tracking in an audit layer

---

Part 5 — LinkedIn Post Template

Post this when you publish the project:

Just published a data engineering project I've been building 🚀

Built a full end-to-end retail POS pipeline completely from scratch using free tools:

🔧 **Stack:**

- Python + Faker → synthetic dirty data generation
- Databricks Free Edition → Bronze/Silver layers with PySpark
- dbt Core → 6 Gold analytical models
- Delta Lake throughout
- 53 automated data quality tests

📊 **What it produces:**

- Daily sales analytics by store, product, and payment method
- Customer lifetime value segmentation
- Inventory stockout risk scoring

- Promotion effectiveness analysis

The whole thing runs at $0/month on free tiers.

Check it out: [link to your GitHub repo]

#DataEngineering #dbt #Databricks #PySpark #DeltaLake #Python #Portfolio

---

Adjust the post to sound like your own voice before posting.

Part 6 — LinkedIn Profile Updates

Headline — add something like:

> Data Analyst | Data Engineering | Python · dbt · Databricks · PySpark

Featured Section — pin your GitHub repo link so it appears at the top of your profile.

Projects Section:

- Title: `SmartRetail POS Intelligence Pipeline`
- Description: End-to-end data pipeline using Python, Databricks, and dbt Core. Medallion architecture (Bronze/Silver/Gold) with Delta Lake and 53 automated data quality tests.
- Link: your GitHub repo URL
